

SMARTCASHIER

DOKUMENTASI & PANDUAN PROYEK

Mata Kuliah: Pemrograman Berorientasi Objek

Kelompok: 7

Anggota Kelompok:

1. Farel Gusviransyah
2. Lutfi Nasrullah Aziz
3. Muhammad Alfin Ramadhan
4. Muhammad Fadli Al Hafizh Wibiksana
5. Ringo Noer Junaedy

Di-generate secara otomatis untuk Kelompok 7

PANDUAN LENGKAP REPLIKASI SMARTCASHIER DARI NOL MENGGUNA

Panduan ini berisi instruksi langkah-demi-langkah (step-by-step) untuk membangun ulang seluruh proyek **SMARTCASHIER** dari nol dengan fokus pengembangan backend menggunakan **Apache NetBeans IDE** (versi 12+).

STRUKTUR FOLDER AKHIR PROYEK

Setelah seluruh langkah diikuti, struktur folder proyek akan terlihat sebagai berikut:

```
smartcashier-project/
```

schem

1. Jalankan PostgreSQL Server Anda.

2. Buah

id

id

10

ic

id

INSERT

Halaman 3/12

```
INSERT INTO product (product_name, category, quantity, price) VALUES
```

LANGKAH 2: MEMBUAT PROYEK BACKEND DI NETBEANS

1. Inisialisasi Proyek Maven Baru

1. Buka **Apache NetBeans**.

> **[!IMPORTANT]**

4. Isi rincian proyek:

2. Konfigurasi `pom.xml`

1. Di panel **Projects** sebelah kiri, buka folder proyek `backend` -> double-click file **`pom.xml`**.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<groupId>com.smartcashier</groupId>
```

```
<properties>
```

```
<dependencies>
```

```
<build>
```

3. Simpan file (`Ctrl + S`).

4. Klik
Boot k

3. Membuat Application Properties

1. Masuk to tab **Files** (sebelah tab Projects) atau ekspansi **Other Sources** -> **src/main/resources**.

2. Jika
Name

```
spring.datasource.url=jdbc:postgresql://localhost:5432/smartcashier_db
```

spring

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jackson.time-zone=Asia/Jakarta
```

4. Membuat Packages Program

Di bawah folder **Source Packages** -> `com.smartcashier`, buat beberapa sub-package dengan cara: klik kanan `com.smartcashier` -> **New** -> **Java Package**.

LANGKAH 3: MENULIS KODE PROGRAM JAVA (BACKEND)

Buat class Java di package masing-masing dengan cara: klik kanan package -> **New** -> **Java Class**.

1. Main Application Class

Buat class `SmartcashierApplication.java` di dalam package `com.smartcashier`:

```
package com.smartcashier;
```

```
import org.springframework.boot.SpringApplication;
```

```
@SpringBootApplication
```

2. Entity Classes (`com.smartcashier.entity`)

- User.java (Abstract Class)**

```
package com.smartcashier.entity;
```

```
import jakarta.persistence.MappedSuperclass;
```

```
@MappedSuperclass
```

@Gette

```
public User() {}
```

pu

- ****Karyawan.java****

```
package com.smartcashier.entity;
```

```
import jakarta.persistence.Entity;
```

import t

```
@Entity
```

@Table

```
public Karyawan() {}
```

pu

```
public boolean login(String inputId, String inputPassword) {
```

```
public void addKpi(double score) {
```

```
@Override
```

pu

- ****Member.java****

```
package com.smartcashier.entity;  
import jakarta.persistence.Entity;
```

import

```
@Entity
```

@Table

```
public Member() {}
```

```
@Override
```

pu

- ****Product.java****

```
package com.smartcashier.entity;  
import jakarta.persistence.*;
```

import

```
@Entity
```

@Table

- ****Transaction.java****

```
package com.smartcashier.entity;  
import com.smartcashier.service.CalculatorTools;
```

import

```
@Entity
```

@Table

@ManyToOne

@ManyToOne

private Double totalAmount;

@OneToMany(mappedBy = "transaction", cascade = CascadeType.ALL, orphanRemoval = true)

public void calculateTotal() {

if (member != null) {

• ****TransactionItem.java****

package com.smartcashier.entity;

import com.fasterxml.jackson.annotation.JsonIgnore;

@Entity

@ManyToOne

@ManyToOne

private Integer quantity;

3. Jpa Repositories (`com.smartcashier.repository`)

- ****KaryawanRepository.java****

```
package com.smartcashier.repository;
```

```
import com.smartcashier.entity.Karyawan;
```

```
public interface KaryawanRepository extends JpaRepository<Karyawan, String> {
```

- ****MemberRepository.java****

```
package com.smartcashier.repository;
```

```
import com.smartcashier.entity.Member;
```

```
public interface MemberRepository extends JpaRepository<Member, String> {
```

- Buat file serupa `ProductRepository.java` dan `TransactionRepository.java` dengan ekstensi `JpaRepository<Entity, Type>` standar.

4. Service Classes (`com.smartcashier.service`)

- ****CalculatorTools.java****

```
package com.smartcashier.service;
```

```
public class CalculatorTools {
```

- ****ProductService.java****

```
package com.smartcashier.service;
```

```
import com.smartcashier.entity.Product;
```

```
@Service
```

```
public synchronized void decreaseStock(Integer productId, int quantity) {
```

- Buat service lainnya (`TransactionService.java`, `KaryawanService.java`, `MemberService.java`) untuk melayani transaksi belanja kasir dan penyimpanan data member ke database.

5. DTO & Controller Classes (`com.smartcashier.dto` & `controller`)

Buat format **Request** dan **Response** data pada DTO, lalu hubungkan ke REST Controller:

- **A**

— — —

LANGKAH 4: MEMBANGUN FRONTEND REACT.JS (MENGUNAKAN VITE SEBAGAI BUNDLER)

> [!NOTE]

.. **Vit

code p

1. Buka Terminal CMD atau PowerShell di luar NetBeans.

1. Buah

...

LANGKAH 5: RUN & DEBUG APLIKASI DI NETBEANS

1. ****Menjalankan Program****:

Kli

